API TESTING

PYRAMID

Practical Payment API examples • High-contrast code • Interview preparation

UNIT → COMPONENT → INTEGRATION → END-TO-END

A practical guide for API automation and senior SDET interviews

How to use this guide

This guide uses a Payment Service as a running example. The same principles apply to order, customer, booking, insurance and other microservice-based APIs.

Key idea
Put most tests at the bottom of the pyramid, where they are fast and focused. Keep fewer tests at the top, where they validate complete business journeys but cost more to maintain.

The code blocks in this edition use dark, high-contrast text on a light background so code remains readable when viewed on screen or printed.

1. What is the Testing Pyramid?

The testing pyramid is a strategy for distributing automated tests across different levels. It encourages many fast, focused tests and fewer broad end-to-end tests.

END-TO-END TESTS	Few • slower • full business flow
INTEGRATION TESTS	Service-to-service • infrastructure
COMPONENT TESTS	One service • dependencies controlled
UNIT TESTS	Many • fast • isolated

How to read it: Unit tests form the wide base because they are quick and inexpensive. Moving upward, tests cover more components and business behaviour, but generally become slower, more complex and more expensive to maintain.

Running example: Payment API

Assume POST /api/payments validates a request, checks customer eligibility, calls a payment gateway, stores the transaction and returns a payment status.

Sample request — POST /api/payments

```
{
  "customerId": "CUST-1001",
  "amount": 250.0,
  "currency": "INR",
  "paymentMethod": "CARD"
}
```

Sample response — HTTP 201 Created

```
{
  "paymentId": "PAY-9001",
  "customerId": "CUST-1001",
  "amount": 250.0,
  "currency": "INR",
  "status": "SUCCESS"
}
```

2. Unit Testing — test the smallest logic

A unit test checks one method or class in isolation. Databases, HTTP services and payment gateways are not called.

Payment API example: Validate amount, currency, fee calculation or payment-status mapping without starting the API server.

Why use it? It finds logic defects early, runs quickly, is easy to debug, and supports many input combinations including edge cases.

Java / JUnit-style unit test

```
public class PaymentValidator {
    public boolean isValidAmount(double amount) {
        return amount > 0 && amount <= 100000;
    }
}

@Test
void shouldRejectZeroAmount() {
    PaymentValidator validator = new PaymentValidator();
    assertFalse(validator.isValidAmount(0));
}
```

Typical test cases: positive amount; zero; negative amount; amount above the allowed limit; unsupported currency.

Scope boundary
A unit test does not prove that the real payment gateway accepts a card. That interaction belongs at another testing level.

3. Component Testing — test one API service

Component testing runs one application component or microservice, including its API endpoint and internal behaviour. External dependencies are controlled with mocks or stubs.

Payment API example: Start the Payment Service and call POST /api/payments. Use WireMock to simulate Customer Service and the payment gateway. Use a test database or repository setup appropriate to the test.

Why use it? It validates routing, request parsing, validation, business logic and response mapping together while avoiding unstable external dependencies.

Request sent by the API test

```
{
  "customerId": "CUST-1001",
  "amount": 250.0,
  "currency": "INR",
  "paymentMethod": "CARD"
}
```

Expected API response

```
{
  "paymentId": "PAY-9001",
  "status": "SUCCESS",
  "message": "Payment processed successfully"
}
```

REST Assured component-test example

```
given()
    .contentType(ContentType.JSON)
    .body(paymentRequest)
.when()
    .post("/api/payments")
.then()
    .statusCode(201)
    .body("status", equalTo("SUCCESS"));
```

Dependency setup: WireMock returns a controlled gateway response. The Payment Service itself is real and receives the HTTP request.

4. Integration Testing — test interactions between systems

Integration tests verify whether two or more services, infrastructure components or external systems communicate correctly.

Payment API example: Verify that Payment Service calls Customer Service correctly, writes payment details to the database and handles gateway responses. Use real dependencies for the interactions under test; isolate unrelated dependencies when useful.

Why use it? It detects incorrect URLs, authentication, headers, request/response mapping, persistence, messaging and service-to-service communication.

Payment request

```
{
  "customerId": "CUST-1001",
  "amount": 250.0,
  "currency": "INR",
  "paymentMethod": "CARD"
}
```

Payment response after gateway interaction

```
{
  "paymentId": "PAY-9001",
  "status": "SUCCESS",
  "gatewayReference": "GW-778812"
}
```

Example assertions: Payment Service receives the customer response; gateway request contains the correct amount and currency; database stores the expected status; API returns the mapped gateway result.

Important
If the gateway is mocked, you can test integration between your service and the mock boundary. You are not proving the real gateway works; use the provider sandbox for that.

5. End-to-End Testing — test the complete business journey

An end-to-end test validates a complete user or business flow across the system, using as many real components as practical and a sandbox for external providers when needed.

Payment API example: Create an order, submit payment, validate the customer, process payment through the gateway sandbox, update the order and verify final confirmation.

Why use it? It confirms critical business journeys, real configuration, authentication, routing and data flow.

Illustrative end-to-end flow

1. Create order	2. Submit payment	3. Validate customer
Order API creates an order.	Payment API receives the request.	Customer Service confirms eligibility.
4. Process gateway	**5. Update order**	**6. Confirm**
Gateway sandbox authorises payment.	Order becomes PAID.	Confirmation is returned.

Expected final business outcome

```
{
  "orderId": "ORD-5001",
  "paymentId": "PAY-9001",
  "orderStatus": "PAID",
  "paymentStatus": "SUCCESS",
  "confirmation": "Order confirmed"
}
```

Keep E2E focused
Cover the most important business journeys and a small number of critical failure paths. Avoid duplicating every validation already covered by unit and component tests.

6. Quick comparison — when and why to use each level

Choose the level based on what you need to prove.

Level	Scope	Dependencies	Speed / cost	Main purpose
Unit	Method or class	None / mocked	Fast / low	Validate business logic
Component	One running service/API	External dependencies mocked	Fast–medium / moderate	Validate service behaviour
Integration	Interactions across systems	Real or selectively mocked	Medium–slow / higher	Validate communication and data flow
End-to-End	Complete business journey	Mostly real; sandbox if needed	Slow / highest	Validate critical user outcomes

Illustrative test distribution

Unit	**65%**	Largest share: quick feedback and broad logic coverage.
Component	**20%**	Service/API behaviour with controlled dependencies.
Integration	**10%**	Important service and infrastructure interactions.
End-to-End	**5%**	Critical business journeys only.

These percentages are illustrative, not mandatory. The right distribution depends on architecture, risk, delivery speed and maintenance cost.

7. Interview-ready answer

Use this as a spoken answer, adapting it to your own project.

Sample answer

In API automation, I apply the testing pyramid by keeping most tests at the unit and component levels, with fewer integration tests and a small number of end-to-end tests. For a Payment API, unit tests validate individual rules such as amount and currency validation. Component tests call the running Payment Service API while mocking external dependencies such as the payment gateway. Integration tests verify communication between the Payment Service, Customer Service and database. End-to-end tests validate the complete business journey, such as creating an order, processing payment and confirming the order. This approach gives fast feedback, isolates failures and controls test execution and maintenance costs.

8. Common interview follow-up questions

Why should we not put everything in E2E tests?

E2E tests are slower, more expensive to maintain and can fail because of environment or dependency issues. Lower-level tests give faster and more precise feedback.

Can WireMock be used in integration testing?

Yes. It can isolate an external provider while you test integration between your own services. Be clear that this does not verify the real provider.

What is the difference between component and integration testing?

Here, component testing validates one service as a running unit with external dependencies controlled. Integration testing validates communication between multiple components or infrastructure elements. Terminology can vary by organisation.

Where do contract tests fit?

Contract tests verify that a service provider and consumer agree on request and response structures. They complement component and integration tests; they do not replace end-to-end tests.

Should all tests use a real database?

No. Use the dependency strategy appropriate to the level. Unit tests usually avoid a database; component tests may use an isolated test database; integration tests verify real database behaviour when that interaction is under test.

9. Practical API automation checklist

Use this checklist when designing or reviewing an API test strategy.

✓ Unit: cover validation, calculations, mapping and decision logic.

✓ Component: test status codes, schema, headers, business rules and error handling with controlled dependencies.

✓ Integration: verify service calls, database persistence, messaging and authentication between systems.

✓ End-to-End: cover critical business journeys and high-value failure paths.

✓ Use deterministic test data, clear cleanup, meaningful assertions and useful reports.

✓ Avoid duplicating every low-level validation in slow E2E tests.

Final takeaway
Test logic at the bottom, service behaviour in the middle, and critical business journeys at the top. The pyramid balances confidence, speed and maintenance; it is not a fixed-percentage rule.